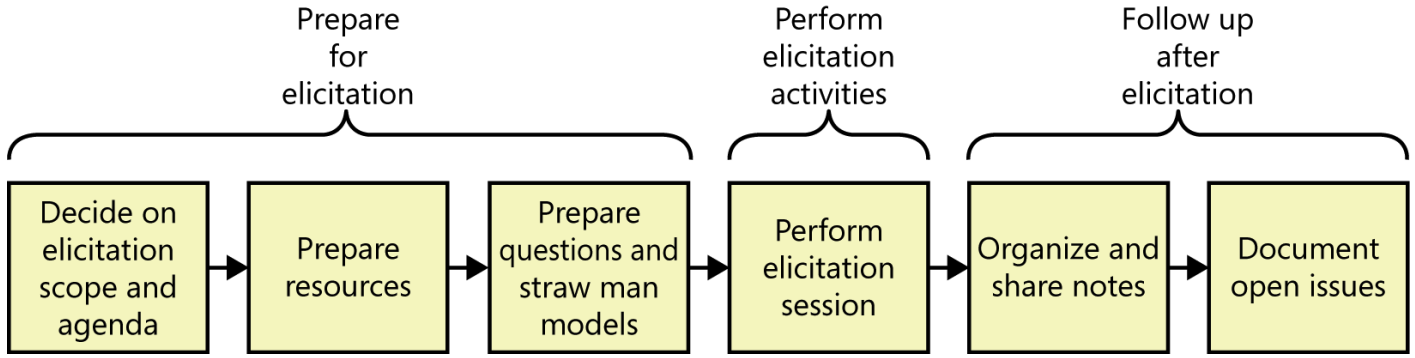
Software Requirements Engineering

Lecture 6: Software Requirements Analysis

Review

1. Requirements elicitation process
2. Software requirements elicitation techniques



Outline

1. Understanding user requirements
2. Setting requirement priorities

Understanding user requirements

- Requirements analysis involves refining the requirements to ensure that all stakeholders understand them and scrutinizing them for errors, omissions, and other deficiencies.
- Analysis includes :
  - decomposing high-level requirements into appropriate levels of detail
  - building prototypes
  - evaluating feasibility
  - negotiating priorities.
- The goal is to develop requirements of sufficient quality and precision that managers can construct realistic project estimates and technical staff can proceed with design, construction, and testing.

Good practices

- Model the application environment
- Create user interface and technical prototypes
- Analyze requirement feasibility
- Prioritize the requirements
- Create a data dictionary
- Model the requirements
- Analyze interfaces between your system and the outside world
- Allocate requirements to subsystems

Good practices

- Model the application environment
  - The context diagram is a simple analysis model that shows how the new system fits into its environment.
  - It defines the boundaries and interfaces between the system being developed and external entities, such as users, hardware devices, and other systems.
  - An ecosystem map shows the various systems in the solution space that interact with each other and the nature of their interconnections.
- Create user interface and technical prototypes
  - Prototypes allow developers and users to achieve a mutual understanding of the problem being solved, as well as helping to validate requirements.

Good practices

- Analyze requirement feasibility
  - Work with developers to evaluate the feasibility of implementing each requirement at acceptable cost and performance in the intended operating environment.
  - Understand the risks associated with implementing each requirement.
  - Requirements that are technically infeasible or overly expensive to implement can perhaps be simplified and still contribute to achieving the project's business objectives.
- Prioritize the requirements
  - Prioritize requirements to ensure that the team implements the highest value or most timely functionality first.
  - Determine which release will contain each feature or set of requirements.
  - Adjust priorities throughout the project as new requirements are proposed and as customer needs, market conditions, and business goals evolve

Good practices

- Create a data dictionary
  - Definitions of the data items and structures associated with the system reside in the data dictionary.
  - This enables everyone working on the project to use consistent data definitions.
- Model the requirements
  - An analysis model is a diagram that depicts requirements visually
  - Models can reveal incorrect, inconsistent, missing, and superfluous requirements.
  - Such models include data flow diagrams, entity-relationship diagrams, state-transition diagrams, state tables, dialog maps, decision trees, and others.

Good practices

- Analyze interfaces between your system and the outside world
  - All software systems have connections to other parts of the world through external interfaces.
  - Analyzing these helps make sure that your application will fit smoothly into its environment.
- Allocate requirements to subsystems
  - The requirements for a complex product that contains multiple subsystems must be apportioned among the various software, hardware, and human subsystems and components.

Use Cases and User Stories

- A **use case** describes a sequence of interactions between a system and an external actor that results in the actor being able to achieve some outcome of value.
- A **user story** is a short, simple description of a feature told from the perspective of the person who desires the new capability.
  - As a <type of user>, I want <some goal> so that <some reason>.

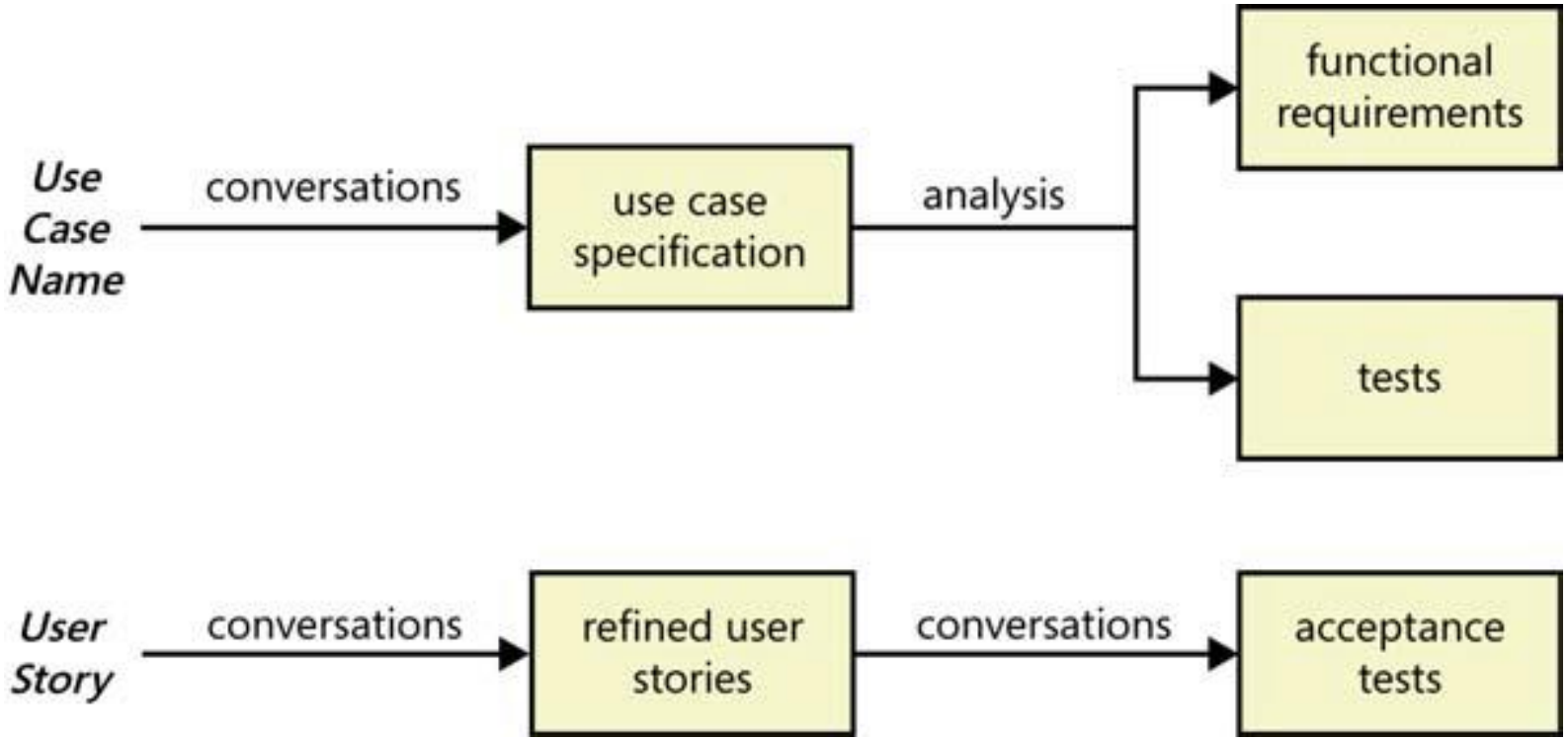
Use Cases

Application	Sample use case
Chemical tracking system	Request a Chemical Print Material Safety Data Sheet Change a Chemical Request Check Status of an Order Generate Quarterly Chemical-Usage Reports
Airport check-in kiosk	Check in for a Flight Print Boarding Passes Change Seats Check Luggage Purchase an Upgrade
Accounting system	Create an Invoice Reconcile an Account Statement Enter a Credit Card Transaction Print Tax Forms for Vendors Search for a Specific Transaction
Online bookstore	Update Customer Profile Search for an Item Buy an Item Track a Shipped Package Cancel an Unshipped Order

Use Cases and User Stories

Application	Sample use case	Corresponding user story
Chemical tracking system	Request a Chemical	As a chemist, I want to request a chemical so that I can perform experiments.
Airport check-in kiosk	Check in for a Flight	As a traveler, I want to check in for a flight so that I can fly to my destination.
Accounting system	Create an Invoice	As a small business owner, I want to create an invoice so that I can bill a customer.
Online bookstore	Update Customer Profile	As a customer, I want to update my customer profile so that future purchases are billed to a new credit card number.

Use Cases and User Stories



How user requirements lead to functional requirements and tests with the use case approach and the user story approach.

Use Cases and User Stories

- Use cases dive further into describing how the user imagines interacting with the system to accomplish his objective.
- Use case should not get into design specifics.

- User stories provide a concise statement of a user's needs.
- User stories offer the advantage of simplicity and conciseness, but there is a tradeoff.

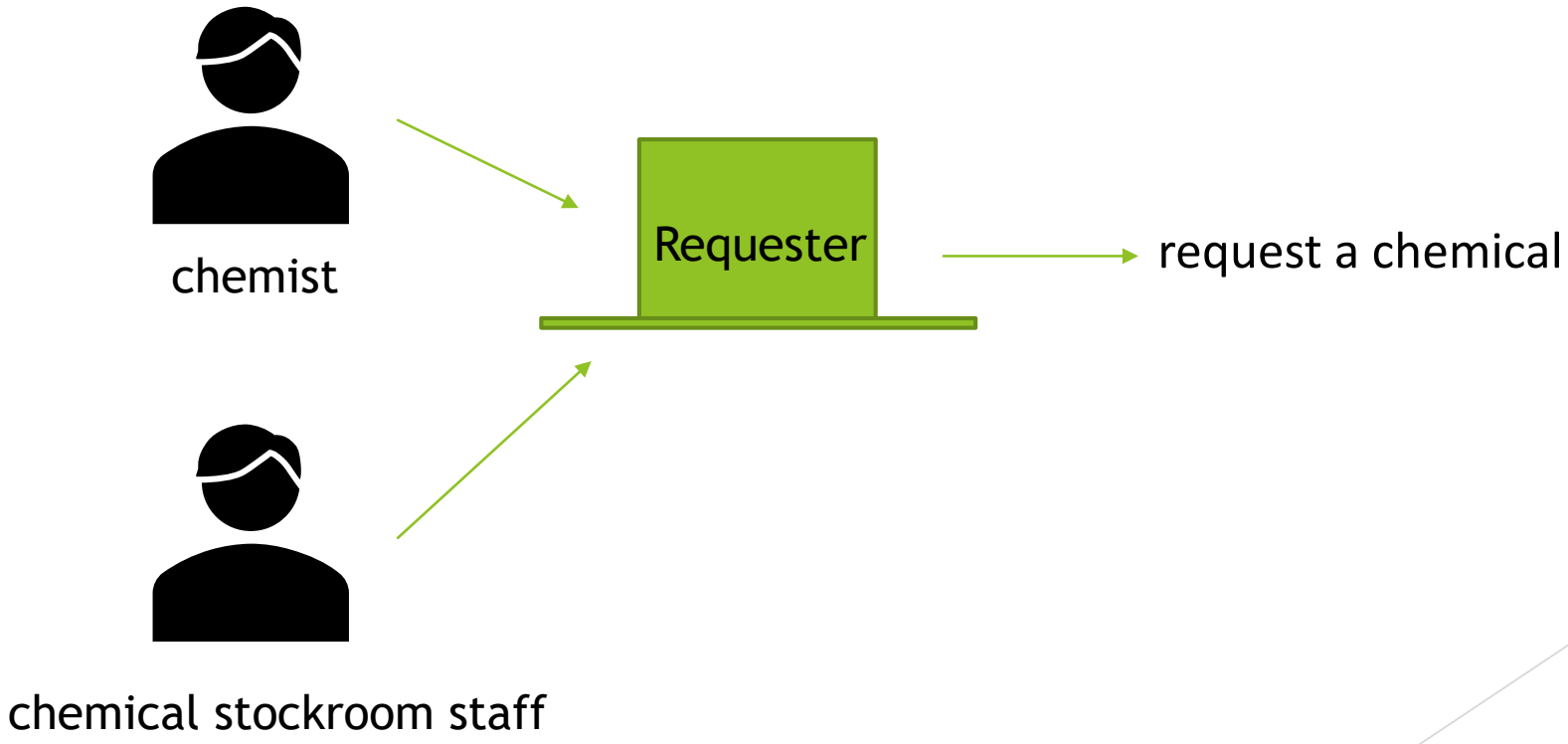
- They provide an organized way for the BA to lead elicitation discussions.

The Use Case Approach

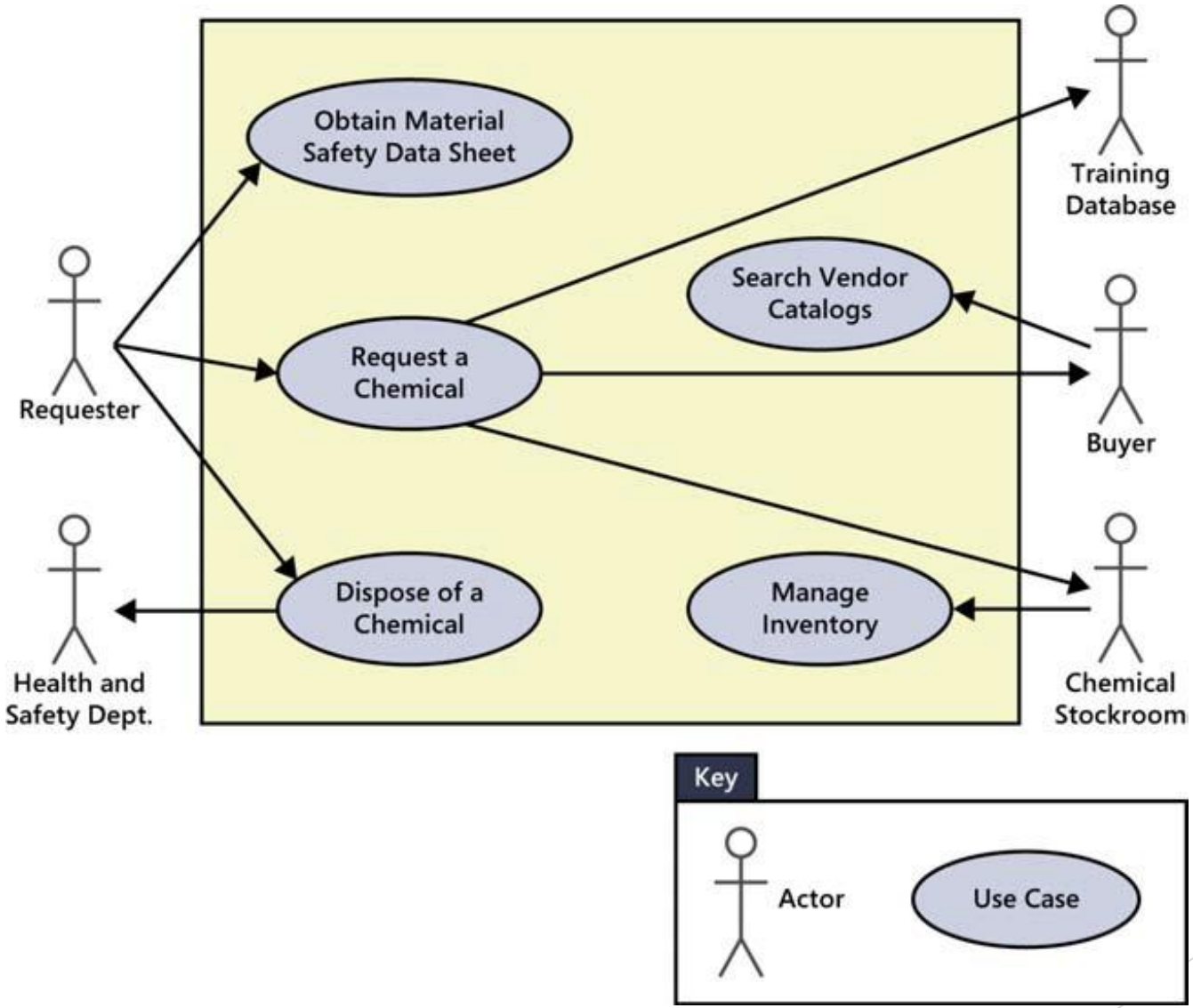
- A use case describes a sequence of interactions between a system and an external actor.
- An actor is a person (or sometimes another software system or a hardware device) that interacts with the system to perform a use case.
- Following are some questions you might ask to help user representatives identify actors:
  1. Who (or what) is notified when something occurs within the system?
  2. Who (or what) provides information or services to the system?
  3. Who (or what) helps the system respond to and complete a task?

Users and Actors

- A user can have different identities in different use cases.



Use Case Diagram



Use Cases and Usage Scenarios

- A use case describes a discrete, standalone activity that an actor can perform to achieve some outcome of value.
- A scenario is a description of a single instance of usage of the system.

=> A use case is therefore a collection of related usage scenarios, and a scenario is a specific instance of a use case.

Use cases

The essential elements of a use case are the following:

- A unique identifier and a succinct name that states the user goal
- A brief textual description that describes the purpose of the use case
- A trigger condition that initiates execution of the use case
- Zero or more preconditions that must be satisfied before the use case can begin
- One or more postconditions that describe the state of the system after the use case is successfully completed
- A numbered list of steps that shows the sequence of interactions between the actor and the system—a dialog—that leads from the preconditions to the postconditions

Preconditions and postconditions

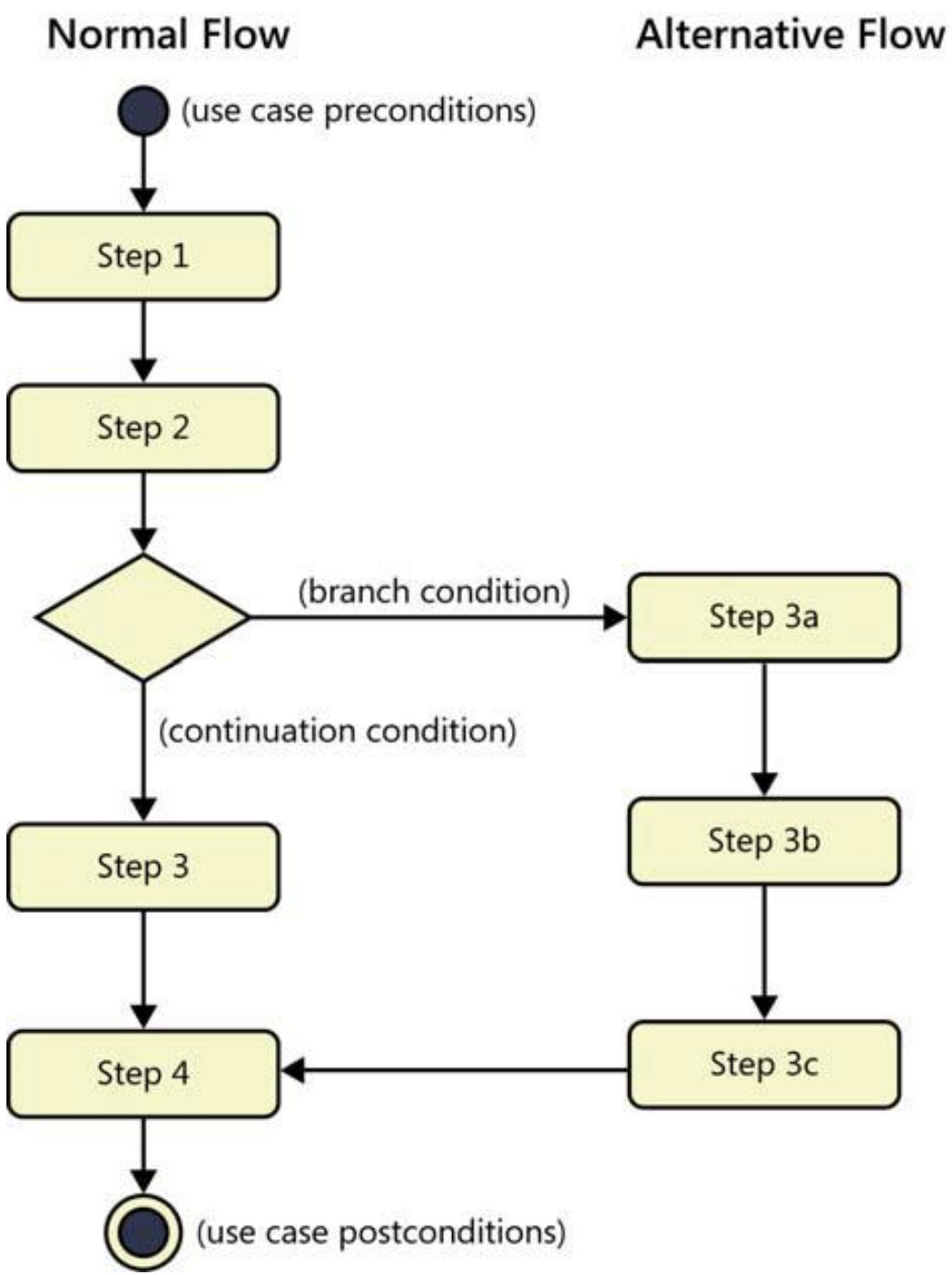
- Preconditions define prerequisites that must be met before the system can begin executing the use case.
- Preconditions could describe the system state, but they don't describe the user's intent.

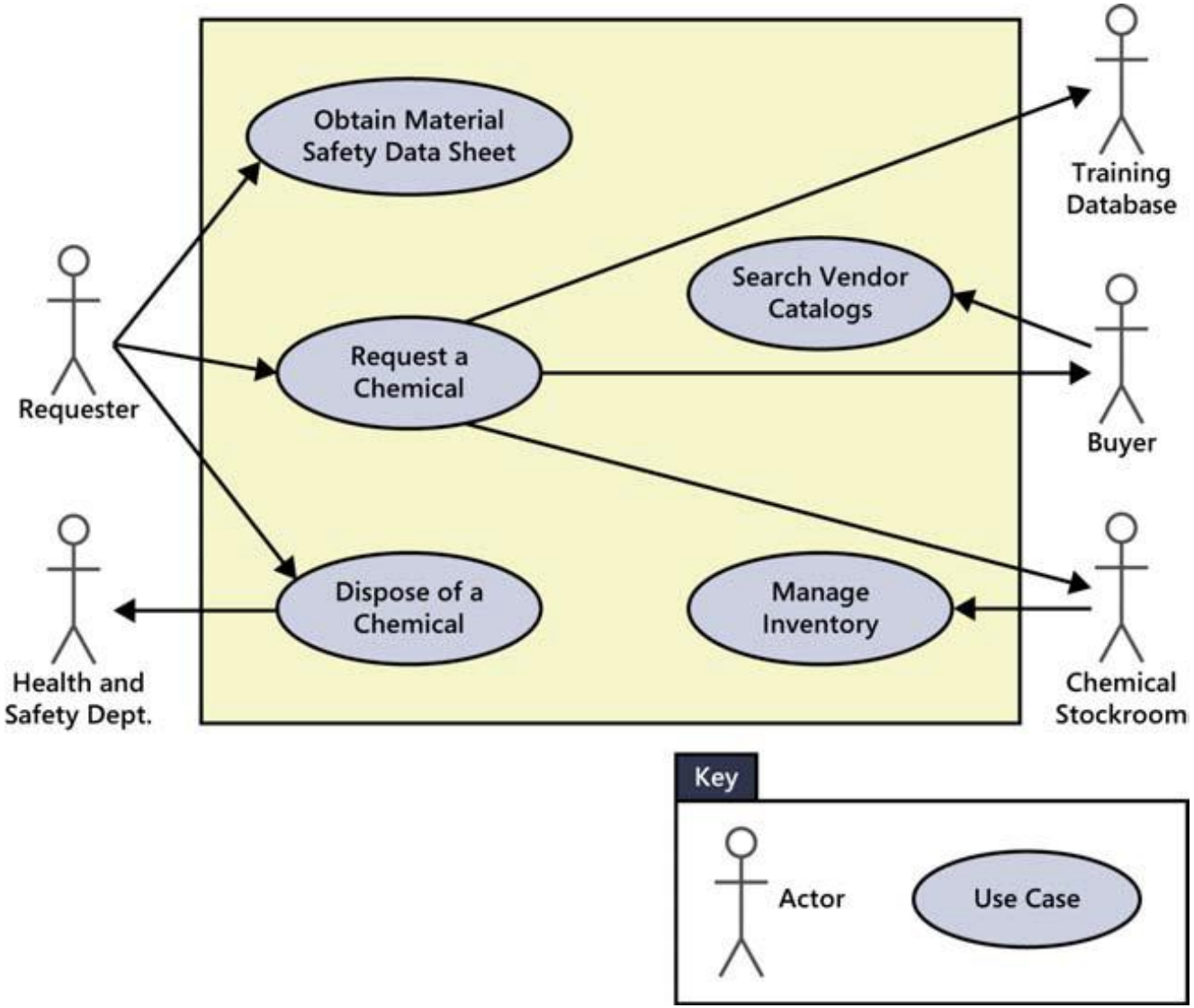
- Postconditions describe the state of the system after the use case executed successfully.
- Postconditions can describe:
  - Something observable to the user
  - Physical outcomes
  - Internal system state changes

Normal flows, alternative flows, and exceptions

- One scenario is identified as the normal flow or main flow of events for the use case.
- Other success scenarios within the use case are called alternative flows or secondary scenarios.
- Conditions that have the potential to prevent a use case from succeeding are called exceptions.

- No every alternative flow needs to be implemented.
- Must implement the exceptions that can prevent the flows that you do implement from succeeding


Normal Flow
Alternative Flow
(use case preconditions)
Step 1
Step 2
(branch condition)
Step 3a
(continuation condition)
Step 3
Step 3b
Step 4
Step 3c
(use case postconditions)



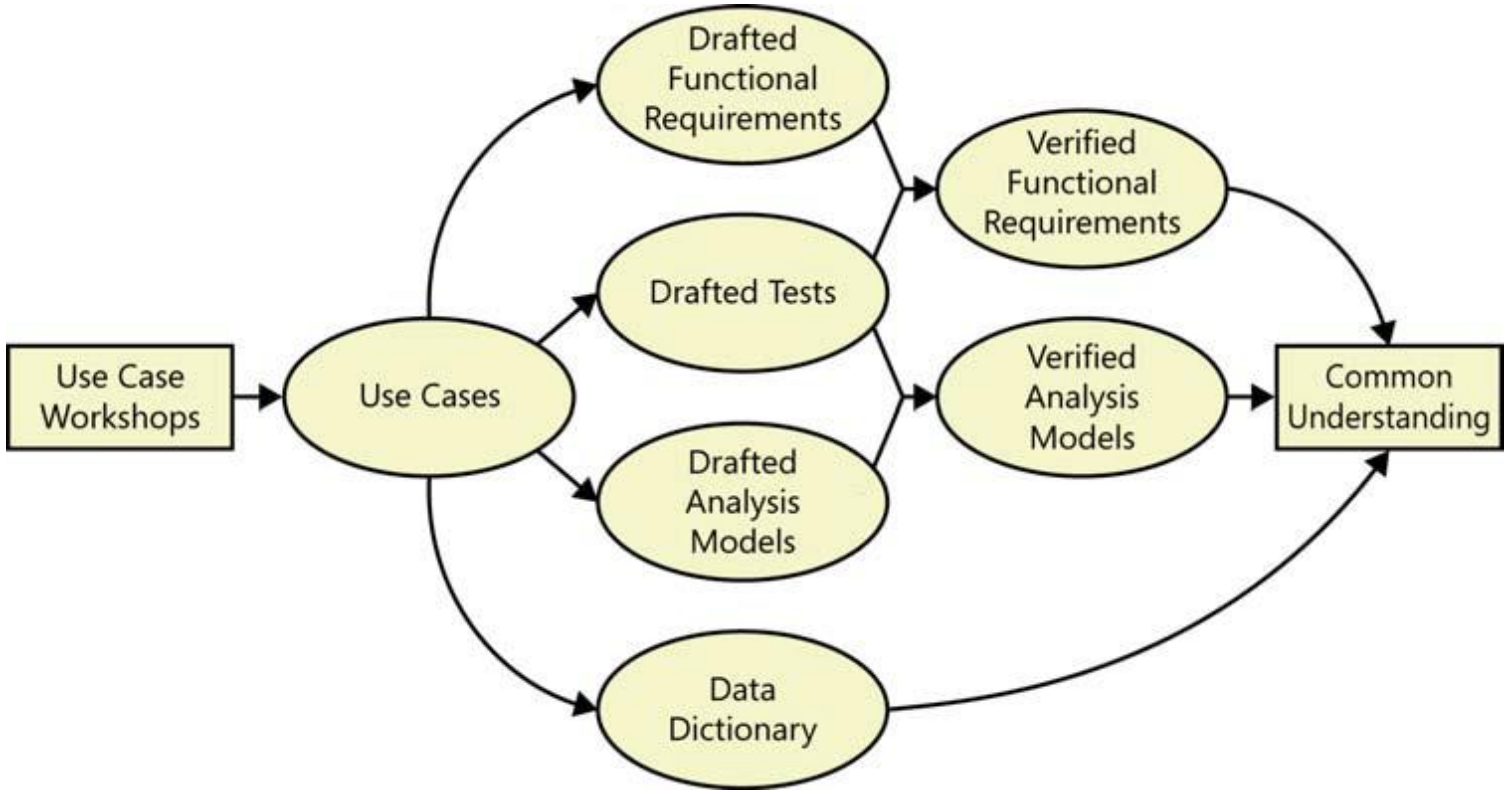
Some of the steps in an alternative flow will be the same as those in the normal flow, but certain unique actions are needed to accomplish the alternative path.

Identifying use cases

You can identify use cases in several ways:

- Identify the actors first, then lay out the business processes being supported by the system, and define the use cases for activities where actors and systems interact.
- Create a specific scenario to illustrate each business process, then generalize the scenarios into use cases and identify the actors involved in each one.
- Using a business process description, ask, “What tasks must the system perform to complete this process or convert the inputs into outputs?” Those tasks might be use cases.
- Identify the external events to which the system must respond, then relate these events to participating actors and specific use cases.
- Identify data entities that require use cases to create, read, update, delete, or otherwise manipulate them.
- Examine the context diagram and ask, “What objectives do each of these external entities want to achieve with the help of the system?”

Exploring Use Cases



Use case elicitation work products

Use cases and functional requirements

- Use cases don’t contain all the information that a developer needs to write the software.
- Document the functionality associated with a use case:
  - Use cases only
  - Use cases and functional requirements
  - Functional requirements only
  - Use cases and tests

Use Case Traps to Avoid

Watch out for the following traps:

- Too many use cases
- Highly complex use cases
- Including design in the use cases
- Including data definitions in the use cases
- **Use cases that users don’t understand**

Setting Requirement Priorities

- Every project with resource limitations needs to define the relative priorities of the requested product capabilities.
- Prioritization helps reveal competing goals, resolve conflicts, plan for staged or incremental deliveries, control scope creep, and make the necessary trade-off decisions.

Setting Requirement Priorities

The successful prioritization requires an understanding of six issues:

1. The needs of the customers
2. The relative importance of requirements to the customers
3. The timing at which capabilities need to be delivered
4. Requirements that serve as predecessors for other requirements and other relationships among requirements
5. Which requirements must be implemented as a group
6. The cost to satisfy each requirement

In the next lecture…

- We will talk about requirements specification.

Question session